

KE CARMEL CENTRAL SCHOOL MUHAMMA



COMPUTER PROJECT

ACADEMIC SESSION : 2026-2027

Project on:

BloodLink: Blood Bank Management System

NAME : S Sahasrajith

CLASS : XII MATRIX

REG. NO. :

CERTIFICATE

K.E. CARMEL CENTRAL SCHOOL MUHAMMA



This is to certify that **S Sahasrajith** (Roll No: _____), has successfully completed the project titled “**BloodLink - Blood Bank Management System**” in the subject Computer Science (083) laid down in the regulations of CBSE for the purpose of practical examination in class XII to be held in **K. E. Carmel Central School, Muhamma.**

Rev. Fr. Scaria Ethirett CMI
Principal

Internal Examiner

External Examiner

DECLARATION

I hereby declare that the Project entitled "**BloodLink - Blood Bank Management System**" is my original work carried out by me as a part of the academic requirements for Class XII MATRIX, Academic Year 2026–2027.

The information, observations, analysis, and conclusions presented in this project are based on my study and experimental work. The project has been completed under the guidance and supervision of my Physics teachers.

Name: **S Sahasrajith**

Class: **XII MATRIX**

Date:

ACKNOWLEDGEMENT

This project, entitled “**BloodLink - Blood Bank Management System**”, is done as per the CBSE syllabus which is a part of my curriculum.

I thank the Almighty for showering His blessings upon me to complete the project successfully on time.

I am also thankful to our honorable principal, **Rev. Fr. Scaria Ethirett CMI**, for giving me the opportunity to do such a project and know more about it.

I express my sincere gratitude to my beloved Computer Science teacher, **Ms. Arunima B.** whose valuable guidance has served as a major contribution towards the completion of the project.

I would finally like to thank my parents, librarian, and all my friends who helped me in bringing out this project successfully.

Name: **S Sahasrajith**

Class: **XII Matrix**

INDEX

Table Of Contents

Serial No.	Heading	Page Number
1	Introduction	6
2	Objective	7
3	Problem Definition	8
4	Reason for Choosing The Topic	9
5	Limitations	10
6	Software Requirements	11
7	Hardware Requirements	12
8	Modules Used	13
9	Functions Used	14
10	Source Code	15
11	Output	38
12	Conclusion	44
13	Bibliography	45

INTRODUCTION

BloodLink is a lightweight desktop application designed to help blood banks efficiently manage donor records, patient records, blood inventory, and blood-related transactions. It provides a simple interface to maintain important information in an organized digital form.

The system uses Python and Tkinter for the graphical user interface and SQLite for local database management. Donor and patient information can be stored, blood donations can be recorded, inventory can be updated, and blood requests can be created and issued while maintaining transaction records.

Built as a practical Computer Science project, BloodLink demonstrates the use of Python programming, functions, decision-making, exception handling, SQL commands, database connectivity, and graphical user interface design to solve a real world management problem.

OBJECTIVE

The objective of this project is to develop a Python and SQL-based system that helps a blood bank maintain donor and patient information and manage blood inventory and transactions. It aims to use database connectivity and a graphical user interface to provide a simple, organized, and interactive experience for managing blood-bank records.

- To design a Python and SQLite-based application that acts as a blood bank management system
- To develop an interactive graphical interface for maintaining donor and patient records .
- To store blood groups and available units efficiently using a local SQL database
- To record donations, create blood requests, and issue blood while maintaining transaction history
- To integrate theoretical knowledge with practical implementation by using Python programming concepts (functions, decision-making, file handling, etc) and database connectivity for real-world problem solving

PROBLEM DEFINITION

Blood banks handle a large amount of information about donors, patients, blood groups, available units, and transactions. Maintaining such records manually can be time-consuming and may make it difficult to quickly find the required information or keep inventory updated.

This project aims to solve this problem by developing an intuitive blood bank management system that stores records digitally and provides the user with functions for donor management, patient management, inventory management, and transaction tracking.

The goal is to create a simple solution that helps organize blood-bank information, reduce repetitive manual work, and make common record-management tasks easier to perform.

REASON FOR CHOOSING THE TOPIC

The topic was chosen because blood banks play an important role in healthcare and require accurate records of blood availability, donors, patients, and requests. Maintaining these details manually can become difficult as the amount of information increases.

By creating a centralized computer-based system, this project provides a practical way to organize these records and demonstrate how programming and database management can be applied to a real-world requirement. The topic is relevant, practical, and useful for understanding how information systems can support organized data management.

LIMITATIONS

1. Manual Data Entry

The system depends on the user to enter correct donor, patient, and transaction information.

2. Local Database

Data is stored in a local SQLite database and is not automatically synchronized between different computers.

3. No Online Connectivity

The current version does not provide an online portal, cloud database, or remote access facility.

4. Basic Validation

The application performs basic input validation but does not replace professional medical or blood-bank verification procedures.

HARDWARE REQUIREMENTS

1. Processor

- a. Minimum : Intel Core i3 or equivalent
- b. Recommended : Intel Core i5 or higher

2. RAM:

- a. Minimum : 4 GB
- b. Recommended : 8 GB or higher

3. Storage :

- a. Minimum : 10 GB
- b. Recommended : 20 GB or higher

4. Display:

- a. Minimum : 1024 × 768 pixels
- b. Recommended : 1366 × 768 pixels

5. Input Devices :

- a. Standard Keyboard and Mouse

SOFTWARE REQUIREMENTS

a. Programming Language

- i. **Python 3.8+**

b. Database:

- i. **sqlite3** – to store donor, patient, inventory and transaction records

c. GUI Toolkit:

- i. **Tkinter** – to create the graphical user interface

d. Code Editor:

- i. IDLE / Visual Studio Code / any Python-compatible editor

e. Operating System:

- i. Windows / Linux / MacOS

MODULES USED

i. **sqlite3**

A built-in Python module used to connect to and manage the local SQLite database.

ii. **tkinter**

The standard Python GUI toolkit used to create windows, forms, buttons, tabs, labels, entry boxes, combo boxes, message boxes and tables.

iii. **ttk**

The themed Tkinter extension used for widgets such as Notebook, Treeview, Button, Frame and Combobox.

iv. **datetime**

Used to retrieve the current date for donation, inventory and transaction records.

FUNCTIONS USED

No.	Function	Purpose
1	connect_db()	Connects to the SQLite database and enables foreign-key support.
2	init_db()	Creates all four tables and initializes the blood-group inventory records.
3	add_donor()	Inserts a new donor record into the database.
4	get_donors()	Retrieves donor records for display in the GUI.
5	add_patient()	Inserts a new patient record into the database.
6	get_patients()	Retrieves patient records for display.
7	get_inventory()	Retrieves the current blood inventory.
8	add_inventory()	Adds units to the selected blood group inventory.
9	record_donation()	Records a donation, updates donor details and increases inventory.
10	create_request()	Creates a pending blood request for a patient.
11	issue_blood()	Checks a pending request, verifies inventory, issues blood and updates the request and inventory.
12	delete_donor()	Deletes a donor when no related transaction exists.
13	delete_patient()	Deletes a patient when no related transaction exists.
14	refresh_all()	Refreshes all tables shown in the application.
15	export()	Retrieves all records from database and creates a downloadable csv file

SOURCE CODE

```
import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import date
```

```
DB_NAME = "BloodB.db"
BLOOD_GROUPS = ("A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-")
```

```
# Connect to the database.
```

```
def connect_db():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

```
# Create table for donors, patients, inventory, and transactions if they do not exist.
```

```
def init_db():
    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS Donors (
            Donor_ID INTEGER PRIMARY KEY AUTOINCREMENT,
            Name TEXT NOT NULL,
            Age INTEGER NOT NULL,
            Gender TEXT,
            Blood_Group TEXT NOT NULL,
            Phone TEXT,
            Address TEXT,
            Last_Donation_Date TEXT,
            Total_Donations INTEGER NOT NULL DEFAULT 0
        )
    """)
```

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS Patients (
        Patient_ID INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        Name TEXT NOT NULL,
        Age INTEGER NOT NULL,
        Gender TEXT,
        Blood_Group_Required TEXT NOT NULL,
        Hospital TEXT,
        Phone TEXT,
        Address TEXT
    )
    """

cur.execute("""
    CREATE TABLE IF NOT EXISTS Blood_Inventory (
        Blood_Group TEXT PRIMARY KEY,
        Available_Units INTEGER NOT NULL DEFAULT 0,
        Last_Updated TEXT
    )
    """)

cur.execute("""
    CREATE TABLE IF NOT EXISTS Blood_Transactions (
        Transaction_ID INTEGER PRIMARY KEY AUTOINCREMENT,
        Transaction_Type TEXT NOT NULL,
        Donor_ID INTEGER,
        Patient_ID INTEGER,
        Blood_Group TEXT NOT NULL,
        Units INTEGER NOT NULL,
        Transaction_Date TEXT NOT NULL,
        Status TEXT NOT NULL,
        FOREIGN KEY (Donor_ID) REFERENCES Donors(Donor_ID) ON DELETE SET NULL,
        FOREIGN KEY (Patient_ID) REFERENCES Patients(Patient_ID) ON DELETE SET NULL
    )
    """)

today = date.today().isoformat()
for group in BLOOD_GROUPS:
    cur.execute("""
        INSERT OR IGNORE INTO Blood_Inventory
        (Blood_Group, Available_Units, Last_Updated)
        VALUES (?, 0, ?)
        """, (group, today))

conn.commit()
conn.close()

```

```

# Check whether a blood group is valid for the system.
def valid_blood_group(group):
    return group.strip().upper() in BLOOD_GROUPS

# Convert a user input into a positive integer and raise an error if invalid.
def positive_int(value, field_name):
    try:
        number = int(value)
    except ValueError:
        raise ValueError(f"{field_name} must be a whole number.")
    if number <= 0:
        raise ValueError(f"{field_name} must be greater than 0.")
    return number

# Add a new donor to the Donors table.
def add_donor(name, age, gender, blood_group, phone, address):
    name = name.strip()
    gender = gender.strip()
    blood_group = blood_group.strip().upper()
    phone = phone.strip()
    address = address.strip()

    if not name:
        raise ValueError("Donor name is required.")
    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")
    age = positive_int(str(age), "Age")
    if age > 120:
        raise ValueError("Enter a realistic age.")
    if not phone:
        raise ValueError("Phone number is required.")

    today = date.today().isoformat()
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO Donors
        (Name, Age, Gender, Blood_Group, Phone, Address, Last_Donation_Date,
        Total_Donations)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (name, age, gender, blood_group, phone, address, today, 0))

```

```
donor_id = cur.lastrowid
conn.commit()
conn.close()
return donor_id
```

Return all donor records from the database.

```
def get_donors():
    conn = connect_db()
    rows = conn.execute("SELECT * FROM Donors ORDER BY Donor_ID").fetchall()
    conn.close()
    return rows
```

Delete a donor record by ID.

```
def delete_donor(donor_id):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("DELETE FROM Donors WHERE Donor_ID = ?", (donor_id,))
    changed = cur.rowcount > 0
    conn.commit()
    conn.close()
    return changed
```

Add a new patient to the Patients table.

```
def add_patient(name, age, gender, blood_group, hospital, phone, address):
    name = name.strip()
    gender = gender.strip()
    blood_group = blood_group.strip().upper()
    hospital = hospital.strip()
    phone = phone.strip()
    address = address.strip()

    if not name:
        raise ValueError("Patient name is required.")
    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")
    age = positive_int(str(age), "Age")
    if age > 120:
        raise ValueError("Enter a realistic age.")
    if not phone:
        raise ValueError("Phone number is required.")
```

```

conn = connect_db()
cur = conn.cursor()
cur.execute("""
    INSERT INTO Patients
    (Name, Age, Gender, Blood_Group_Required, Hospital, Phone, Address)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (name, age, gender, blood_group, hospital, phone, address))
patient_id = cur.lastrowid
conn.commit()
conn.close()
return patient_id

```

Return all patient records from the database.

```

def get_patients():
    conn = connect_db()
    rows = conn.execute("SELECT * FROM Patients ORDER BY Patient_ID").fetchall()
    conn.close()
    return rows

```

Delete a patient record by ID.

```

def delete_patient(patient_id):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("DELETE FROM Patients WHERE Patient_ID = ?", (patient_id,))
    changed = cur.rowcount > 0
    conn.commit()
    conn.close()
    return changed

```

Add blood units to the inventory for a specific blood group.

```

def add_inventory(blood_group, units):
    blood_group = blood_group.strip().upper()
    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")
    units = positive_int(str(units), "Units")

    today = date.today().isoformat()
    conn = connect_db()
    cur = conn.cursor()

```

```

cur.execute("""
    UPDATE Blood_Inventory
    SET Available_Units = Available_Units + ?, Last_Updated = ?
    WHERE Blood_Group = ?
""", (units, today, blood_group))
conn.commit()
conn.close()

```

Update inventory by a positive or negative change in units.

```

def update_inventory(blood_group, units_change):
    blood_group = blood_group.strip().upper()
    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")

    try:
        units_change = int(units_change)
    except (TypeError, ValueError):
        raise ValueError("Units must be a whole number.")

    if units_change == 0:
        return

```

```

today = date.today().isoformat()
conn = connect_db()
cur = conn.cursor()
cur.execute("""
    UPDATE Blood_Inventory
    SET Available_Units = Available_Units + ?, Last_Updated = ?
    WHERE Blood_Group = ?
""", (units_change, today, blood_group))
conn.commit()
conn.close()

```

Return the current blood inventory.

```

def get_inventory():
    conn = connect_db()
    rows = conn.execute("""
        SELECT Blood_Group, Available_Units, Last_Updated
        FROM Blood_Inventory
        ORDER BY CASE Blood_Group
            WHEN 'A+' THEN 1 WHEN 'A-' THEN 2

```

```

        WHEN 'B+' THEN 3 WHEN 'B-' THEN 4
        WHEN 'AB+' THEN 5 WHEN 'AB-' THEN 6
        WHEN 'O+' THEN 7 WHEN 'O-' THEN 8
    END
    """).fetchall()
    conn.close()
    return rows

```

Return all blood transactions in reverse order.

```

def get_transactions():
    conn = connect_db()
    rows = conn.execute("""
        SELECT Transaction_ID, Transaction_Type, Donor_ID, Patient_ID,
               Blood_Group, Units, Transaction_Date, Status
        FROM Blood_Transactions
        ORDER BY Transaction_ID DESC
    """).fetchall()
    conn.close()
    return rows

```

Record a blood donation and update donor and inventory data.

```

def record_donation(donor_id, blood_group, units):
    donor_id = positive_int(str(donor_id), "Donor ID")
    blood_group = blood_group.strip().upper()
    units = positive_int(str(units), "Units")

    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")

    conn = connect_db()
    cur = conn.cursor()

    donor = cur.execute(
        "SELECT Donor_ID, Blood_Group FROM Donors WHERE Donor_ID = ?",
        (donor_id,)
    ).fetchone()

    if donor is None:
        conn.close()
        raise ValueError("Donor ID not found.")

```

```

if donor["Blood_Group"] != blood_group:
    conn.close()
    raise ValueError(
        f"This donor is registered as {donor['Blood_Group']], not {blood_group}."
    )

```

```

today = date.today().isoformat()

```

```

cur.execute("""
    INSERT INTO Blood_Transactions
    (Transaction_Type, Donor_ID, Patient_ID, Blood_Group,
    Units, Transaction_Date, Status)
    VALUES ('DONATION', ?, NULL, ?, ?, ?, 'Completed')
""", (donor_id, blood_group, units, today))

```

```

cur.execute("""
    UPDATE Donors
    SET Last_Donation_Date = ?, Total_Donations = Total_Donations + 1
    WHERE Donor_ID = ?
""", (today, donor_id))

```

```

cur.execute("""
    UPDATE Blood_Inventory
    SET Available_Units = Available_Units + ?, Last_Updated = ?
    WHERE Blood_Group = ?
""", (units, today, blood_group))

```

```

conn.commit()
conn.close()

```

Create a request for blood from a patient.

```

def create_request(patient_id, blood_group, units):
    patient_id = positive_int(str(patient_id), "Patient ID")
    blood_group = blood_group.strip().upper()
    units = positive_int(str(units), "Units")

```

```

    if not valid_blood_group(blood_group):
        raise ValueError("Invalid blood group.")

```

```

    conn = connect_db()
    cur = conn.cursor()

```

```
patient = cur.execute(
    "SELECT Patient_ID FROM Patients WHERE Patient_ID = ?",
    (patient_id,)
).fetchone()
```

```
if patient is None:
    conn.close()
    raise ValueError("Patient ID not found.")
```

```
today = date.today().isoformat()
cur.execute("""
    INSERT INTO Blood_Transactions
    (Transaction_Type, Donor_ID, Patient_ID, Blood_Group,
    Units, Transaction_Date, Status)
    VALUES ('REQUEST', NULL, ?, ?, ?, ?, 'Pending')
""", (patient_id, blood_group, units, today))
```

```
conn.commit()
conn.close()
```

Issue blood for a pending request if enough stock is available.

```
def issue_blood(request_id):
    request_id = positive_int(str(request_id), "Request ID")
```

```
    conn = connect_db()
    cur = conn.cursor()
```

```
    request = cur.execute("""
        SELECT *
        FROM Blood_Transactions
        WHERE Transaction_ID = ?
        AND Transaction_Type = 'REQUEST'
        AND Status = 'Pending'
        """, (request_id,)).fetchone()
```

```
    if request is None:
        conn.close()
        return False, "Pending request not found."
```

```
    inventory = cur.execute("""
        SELECT Available_Units
        FROM Blood_Inventory
```

```

WHERE Blood_Group = ?
""" , (request["Blood_Group"],)).fetchone()

if inventory is None or inventory["Available_Units"] < request["Units"]:
    conn.close()
    return False, "Insufficient blood inventory."

today = date.today().isoformat()

cur.execute("""
    INSERT INTO Blood_Transactions
    (Transaction_Type, Donor_ID, Patient_ID, Blood_Group,
    Units, Transaction_Date, Status)
    VALUES ('ISSUE', NULL, ?, ?, ?, ?, 'Completed')
""", (
    request["Patient_ID"],
    request["Blood_Group"],
    request["Units"],
    today
))

cur.execute("""
    UPDATE Blood_Inventory
    SET Available_Units = Available_Units - ?, Last_Updated = ?
    WHERE Blood_Group = ?
""", (request["Units"], today, request["Blood_Group"]))

cur.execute("""
    UPDATE Blood_Transactions
    SET Status = 'Completed'
    WHERE Transaction_ID = ?
""", (request_id,))

conn.commit()
conn.close()
return True, "Blood issued successfully."

```

```

# Directly issue blood to a patient and remove that patient from the list.
def issue_blood_to_patient(patient_id, blood_group, units):
    patient_id = positive_int(str(patient_id), "Patient ID")
    blood_group = blood_group.strip().upper()
    units = positive_int(str(units), "Units")

```

```

if not valid_blood_group(blood_group):
    raise ValueError("Invalid blood group.")

conn = connect_db()
cur = conn.cursor()

patient = cur.execute(
    "SELECT Patient_ID, Blood_Group_Required FROM Patients WHERE Patient_ID = ?",
    (patient_id,)
).fetchone()

if patient is None:
    conn.close()
    raise ValueError("Patient ID not found.")

if patient["Blood_Group_Required"] != blood_group:
    conn.close()
    raise ValueError(
        f"This patient requires {patient['Blood_Group_Required']}, not {blood_group}."
    )

inventory = cur.execute(
    "SELECT Available_Units FROM Blood_Inventory WHERE Blood_Group = ?",
    (blood_group,)
).fetchone()

if inventory is None or inventory["Available_Units"] < units:
    conn.close()
    return False, "Insufficient blood inventory."

today = date.today().isoformat()

cur.execute("""
    INSERT INTO Blood_Transactions
    (Transaction_Type, Donor_ID, Patient_ID, Blood_Group,
    Units, Transaction_Date, Status)
    VALUES ('ISSUE', NULL, ?, ?, ?, ?, 'Completed')
""", (patient_id, blood_group, units, today))

cur.execute("""
    UPDATE Blood_Inventory
    SET Available_Units = Available_Units - ?, Last_Updated = ?

```

```

WHERE Blood_Group = ?
""", (units, today, blood_group))

cur.execute(
    "DELETE FROM Patients WHERE Patient_ID = ?",
    (patient_id,)
)

conn.commit()
conn.close()
return True, "Blood issued successfully and patient record removed."

# GUI variables used for the main application windows.
root = None
donor_tree = None
patient_tree = None
inventory_tree = None
transaction_tree = None

def create_table(parent, columns):
    frame = ttk.Frame(parent)
    frame.pack(fill="both", expand=True, padx=10, pady=10)

    tree = ttk.Treeview(frame, columns=columns, show="headings")

    for column in columns:
        tree.heading(column, text=column)
        tree.column(column, width=125, anchor="center")

    y_scroll = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
    x_scroll = ttk.Scrollbar(frame, orient="horizontal", command=tree.xview)
    tree.configure(yscrollcommand=y_scroll.set, xscrollcommand=x_scroll.set)

    tree.pack(side="top", fill="both", expand=True)
    x_scroll.pack(side="bottom", fill="x")
    y_scroll.pack(side="right", fill="y")

    return tree

```

```
# Remove all rows from a table view.
```

```
def clear_table(tree):  
    for item in tree.get_children():  
        tree.delete(item)
```

```
# Load database rows into a tree view table.
```

```
def load_table(tree, rows):  
    clear_table(tree)  
    for row in rows:  
        tree.insert("", tk.END, values=tuple(row))
```

```
# Refresh the donor list in the GUI.
```

```
def refresh_donors():  
    load_table(donor_tree, get_donors())
```

```
# Refresh the patient list in the GUI.
```

```
def refresh_patients():  
    load_table(patient_tree, get_patients())
```

```
# Refresh the inventory list in the GUI.
```

```
def refresh_inventory():  
    load_table(inventory_tree, get_inventory())
```

```
# Refresh the transaction list in the GUI.
```

```
def refresh_transactions():  
    load_table(transaction_tree, get_transactions())
```

```
# Create a labeled entry or combobox for a form window.
```

```
def make_labeled_entry(parent, label, row, values=None):  
    ttk.Label(parent, text=label).grid(row=row, column=0, padx=10, pady=7, sticky="w")  
  
    if values is not None:  
        widget = ttk.Combobox(parent, values=values, state="readonly", width=28)  
        if values:  
            widget.set(values[0])  
    else:  
        widget = ttk.Entry(parent, width=31)
```

```

widget.grid(row=row, column=1, padx=10, pady=7)
return widget

# Window to add a donor record.
def donor_window():
    win = tk.Toplevel(root)
    win.title("Add Donor")
    win.resizable(False, False)

    frame = ttk.Frame(win, padding=15)
    frame.pack()

    name = make_labeled_entry(frame, "Name", 0)
    age = make_labeled_entry(frame, "Age", 1)
    gender = make_labeled_entry(frame, "Gender", 2, ("M", "F", "O"))
    blood = make_labeled_entry(frame, "Blood Group", 3, BLOOD_GROUPS)
    phone = make_labeled_entry(frame, "Phone", 4)
    address = make_labeled_entry(frame, "Address", 5)

    def save():
        try:
            donor_id = add_donor(
                name.get(), age.get(), gender.get(), blood.get(),
                phone.get(), address.get()
            )
            refresh_donors()
            win.destroy()
            messagebox.showinfo("BloodLink", f"Donor added successfully.\nDonor ID:
{donor_id}")
        except ValueError as exc:
            messagebox.showerror("Invalid Input", str(exc))
        except sqlite3.Error as exc:
            messagebox.showerror("Database Error", str(exc))

    ttk.Button(frame, text="Add Donor", command=save).grid(
        row=6, column=0, columnspan=2, pady=15
    )

```

```

# Window to add a patient record.
def patient_window():
    win = tk.Toplevel(root)
    win.title("Add Patient")
    win.resizable(False, False)

    frame = ttk.Frame(win, padding=15)
    frame.pack()

    name = make_labeled_entry(frame, "Name", 0)
    age = make_labeled_entry(frame, "Age", 1)
    gender = make_labeled_entry(frame, "Gender", 2, ("M", "F", "O"))
    blood = make_labeled_entry(frame, "Blood Group Required", 3, BLOOD_GROUPS)
    hospital = make_labeled_entry(frame, "Hospital", 4)
    phone = make_labeled_entry(frame, "Phone", 5)
    address = make_labeled_entry(frame, "Address", 6)

    def save():
        try:
            patient_id = add_patient(
                name.get(), age.get(), gender.get(), blood.get(),
                hospital.get(), phone.get(), address.get()
            )
            refresh_patients()
            win.destroy()
            messagebox.showinfo(
                "BloodLink", f"Patient added successfully.\nPatient ID: {patient_id}"
            )
        except ValueError as exc:
            messagebox.showerror("Invalid Input", str(exc))
        except sqlite3.Error as exc:
            messagebox.showerror("Database Error", str(exc))

    ttk.Button(frame, text="Add Patient", command=save).grid(
        row=7, column=0, columnspan=2, pady=15
    )

```

```

# Window to record a blood donation.
def donation_window():
    win = tk.Toplevel(root)
    win.title("Record Donation")
    win.resizable(False, False)

```

```

frame = ttk.Frame(win, padding=15)
frame.pack()

donor_id = make_labeled_entry(frame, "Donor ID", 0)
blood = make_labeled_entry(frame, "Blood Group", 1, BLOOD_GROUPS)
units = make_labeled_entry(frame, "Units", 2)

def save():
    try:
        record_donation(donor_id.get(), blood.get(), units.get())
        refresh_donors()
        refresh_inventory()
        refresh_transactions()
        win.destroy()
        messagebox.showinfo("BloodLink", "Donation recorded successfully.")
    except ValueError as exc:
        messagebox.showerror("Invalid Input", str(exc))
    except sqlite3.Error as exc:
        messagebox.showerror("Database Error", str(exc))

ttk.Button(frame, text="Record Donation", command=save).grid(
    row=3, column=0, columnspan=2, pady=15
)

```

Window to add stock to the blood inventory.

```

def inventory_window():
    win = tk.Toplevel(root)
    win.title("Add Blood Units")
    win.resizable(False, False)

    frame = ttk.Frame(win, padding=15)
    frame.pack()

    blood = make_labeled_entry(frame, "Blood Group", 0, BLOOD_GROUPS)
    units = make_labeled_entry(frame, "Units to Add", 1)

    def save():
        try:
            add_inventory(blood.get(), units.get())
            refresh_inventory()
            win.destroy()

```

```

        messagebox.showinfo("BloodLink", "Inventory updated successfully.")
    except ValueError as exc:
        messagebox.showerror("Invalid Input", str(exc))
    except sqlite3.Error as exc:
        messagebox.showerror("Database Error", str(exc))

```

```

ttk.Button(frame, text="Update Inventory", command=save).grid(
    row=2, column=0, columnspan=2, pady=15
)

```

Window to create a blood request for a patient.

```

def request_window():
    win = tk.Toplevel(root)
    win.title("Create Blood Request")
    win.resizable(False, False)

    frame = ttk.Frame(win, padding=15)
    frame.pack()

    patient_id = make_labeled_entry(frame, "Patient ID", 0)
    blood = make_labeled_entry(frame, "Blood Group", 1, BLOOD_GROUPS)
    units = make_labeled_entry(frame, "Units", 2)

    def save():
        try:
            create_request(patient_id.get(), blood.get(), units.get())
            refresh_transactions()
            win.destroy()
            messagebox.showinfo("BloodLink", "Blood request created successfully.")
        except ValueError as exc:
            messagebox.showerror("Invalid Input", str(exc))
        except sqlite3.Error as exc:
            messagebox.showerror("Database Error", str(exc))

    ttk.Button(frame, text="Create Request", command=save).grid(
        row=3, column=0, columnspan=2, pady=15
    )

```

Window to issue blood against a pending request.

```

def issue_window():
    win = tk.Toplevel(root)

```

```

win.title("Issue Blood")
win.resizable(False, False)

frame = ttk.Frame(win, padding=15)
frame.pack()

request_id = make_labeled_entry(frame, "Request Transaction ID", 0)

def issue():
    try:
        success, message = issue_blood(request_id.get())
        if success:
            refresh_inventory()
            refresh_transactions()
            win.destroy()
            messagebox.showinfo("BloodLink", message)
        else:
            messagebox.showerror("Cannot Issue Blood", message)
    except ValueError as exc:
        messagebox.showerror("Invalid Input", str(exc))
    except sqlite3.Error as exc:
        messagebox.showerror("Database Error", str(exc))

ttk.Button(frame, text="Issue Blood", command=issue).grid(
    row=1, column=0, columnspan=2, pady=15
)

```

Window to issue blood directly to a patient and remove the patient record.

```

def direct_issue_window():
    win = tk.Toplevel(root)
    win.title("Issue Blood to Patient")
    win.resizable(False, False)

    frame = ttk.Frame(win, padding=15)
    frame.pack()

    patient_id = make_labeled_entry(frame, "Patient ID", 0)
    blood = make_labeled_entry(frame, "Blood Group", 1, BLOOD_GROUPS)
    units = make_labeled_entry(frame, "Units", 2)

```

```

def issue():
    try:
        success, message = issue_blood_to_patient(patient_id.get(), blood.get(),
units.get())
        if success:
            refresh_patients()
            refresh_inventory()
            refresh_transactions()
            win.destroy()
            messagebox.showinfo("BloodLink", message)
        else:
            messagebox.showerror("Cannot Issue Blood", message)
    except ValueError as exc:
        messagebox.showerror("Invalid Input", str(exc))
    except sqlite3.Error as exc:
        messagebox.showerror("Database Error", str(exc))

ttk.Button(frame, text="Issue to Patient", command=issue).grid(
    row=3, column=0, columnspan=2, pady=15
)

```

Delete the selected record from a table and refresh the list.

```

def delete_selected(tree, table_name, id_column, refresh_function):
    selected = tree.selection()
    if not selected:
        messagebox.showwarning("BloodLink", "Select a record first.")
        return

```

```

    values = tree.item(selected[0], "values")
    record_id = values[0]

```

```

    if not messagebox.askyesno(
        "Confirm Delete", f"Delete {table_name} ID {record_id}?"
    ):
        return

```

```

    conn = connect_db()
    try:
        cur = conn.cursor()
        cur.execute(
            f"DELETE FROM {table_name} WHERE {id_column} = ?",
            (record_id,)
        )

```

```

if cur.rowcount == 0:
    raise ValueError("Record not found.")
conn.commit()
refresh_function()
messagebox.showinfo("BloodLink", "Record deleted.")
except (sqlite3.Error, ValueError) as exc:
    conn.rollback()
    messagebox.showerror("Error", str(exc))
finally:
    conn.close()

```

Start the main application window and initialize all tables.

```

def main():
    global root, donor_tree, patient_tree, inventory_tree, transaction_tree

    init_db()

    root = tk.Tk()
    root.title("BloodLink - Blood Bank Management System")
    root.geometry("1250x720")
    root.minsize(1000, 600)

    style = ttk.Style(root)
    try:
        style.theme_use("clam")
    except tk.TclError:
        pass

    header = ttk.Frame(root, padding=(15, 10))
    header.pack(fill="x")

    ttk.Label(
        header,
        text="BLOODLINK",
        font=("Arial", 24, "bold")
    ).pack()

    ttk.Label(
        header,
        text="Blood Bank Management System",

```

```

    font=("Arial", 11)
).pack()

notebook = ttk.Notebook(root)
notebook.pack(fill="both", expand=True, padx=15, pady=10)

donor_tab = ttk.Frame(notebook)
patient_tab = ttk.Frame(notebook)
inventory_tab = ttk.Frame(notebook)
transaction_tab = ttk.Frame(notebook)

notebook.add(donor_tab, text="Donors")
notebook.add(patient_tab, text="Patients")
notebook.add(inventory_tab, text="Inventory")
notebook.add(transaction_tab, text="Transactions")

donor_tree = create_table(donor_tab, (
    "Donor_ID", "Name", "Age", "Gender", "Blood_Group",
    "Phone", "Address", "Last_Donation_Date", "Total_Donations"
))

patient_tree = create_table(patient_tab, (
    "Patient_ID", "Name", "Age", "Gender", "Blood_Group_Required",
    "Hospital", "Phone", "Address"
))

inventory_tree = create_table(inventory_tab, (
    "Blood_Group", "Available_Units", "Last_Updated"
))

transaction_tree = create_table(transaction_tab, (
    "Transaction_ID", "Transaction_Type", "Donor_ID", "Patient_ID",
    "Blood_Group", "Units", "Transaction_Date", "Status"
))

buttons = ttk.Frame(root, padding=(10, 5))
buttons.pack(fill="x")

ttk.Button(buttons, text="Add Donor", command=donor_window).grid(
    row=0, column=0, padx=4, pady=4
)
ttk.Button(buttons, text="Add Patient", command=patient_window).grid(
    row=0, column=1, padx=4, pady=4
)

```

```

ttk.Button(buttons, text="Record Donation", command=donation_window).grid(
    row=0, column=2, padx=4, pady=4
)
ttk.Button(buttons, text="Add Inventory", command=inventory_window).grid(
    row=0, column=3, padx=4, pady=4
)
ttk.Button(buttons, text="Create Request", command=request_window).grid(
    row=0, column=4, padx=4, pady=4
)
ttk.Button(buttons, text="Issue Blood", command=issue_window).grid(
    row=0, column=5, padx=4, pady=4
)
ttk.Button(buttons, text="Issue to Patient", command=direct_issue_window).grid(
    row=0, column=6, padx=4, pady=4
)

ttk.Button(
    buttons,
    text="Delete Donor",
    command=lambda: delete_selected(
        donor_tree, "Donors", "Donor_ID", refresh_donors
    )
).grid(row=1, column=0, padx=4, pady=4)

ttk.Button(
    buttons,
    text="Delete Patient",
    command=lambda: delete_selected(
        patient_tree, "Patients", "Patient_ID", refresh_patients
    )
).grid(row=1, column=1, padx=4, pady=4)

ttk.Button(
    buttons,
    text="Refresh All",
    command=lambda: (
        refresh_donors(),
        refresh_patients(),
        refresh_inventory(),
        refresh_transactions()
    )
).grid(row=1, column=2, padx=4, pady=4)

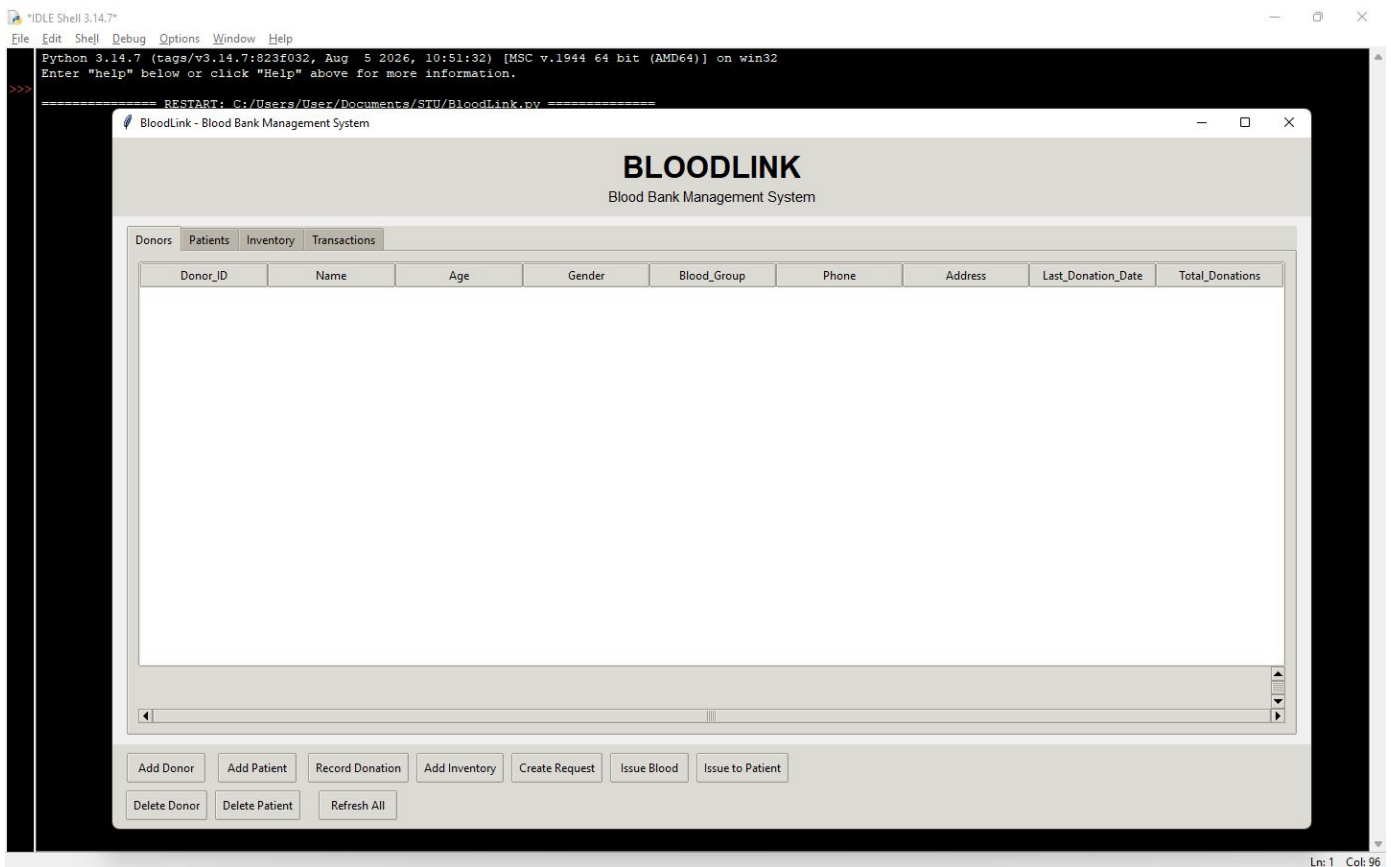
```

```
refresh_donors()
refresh_patients()
refresh_inventory()
refresh_transactions()
```

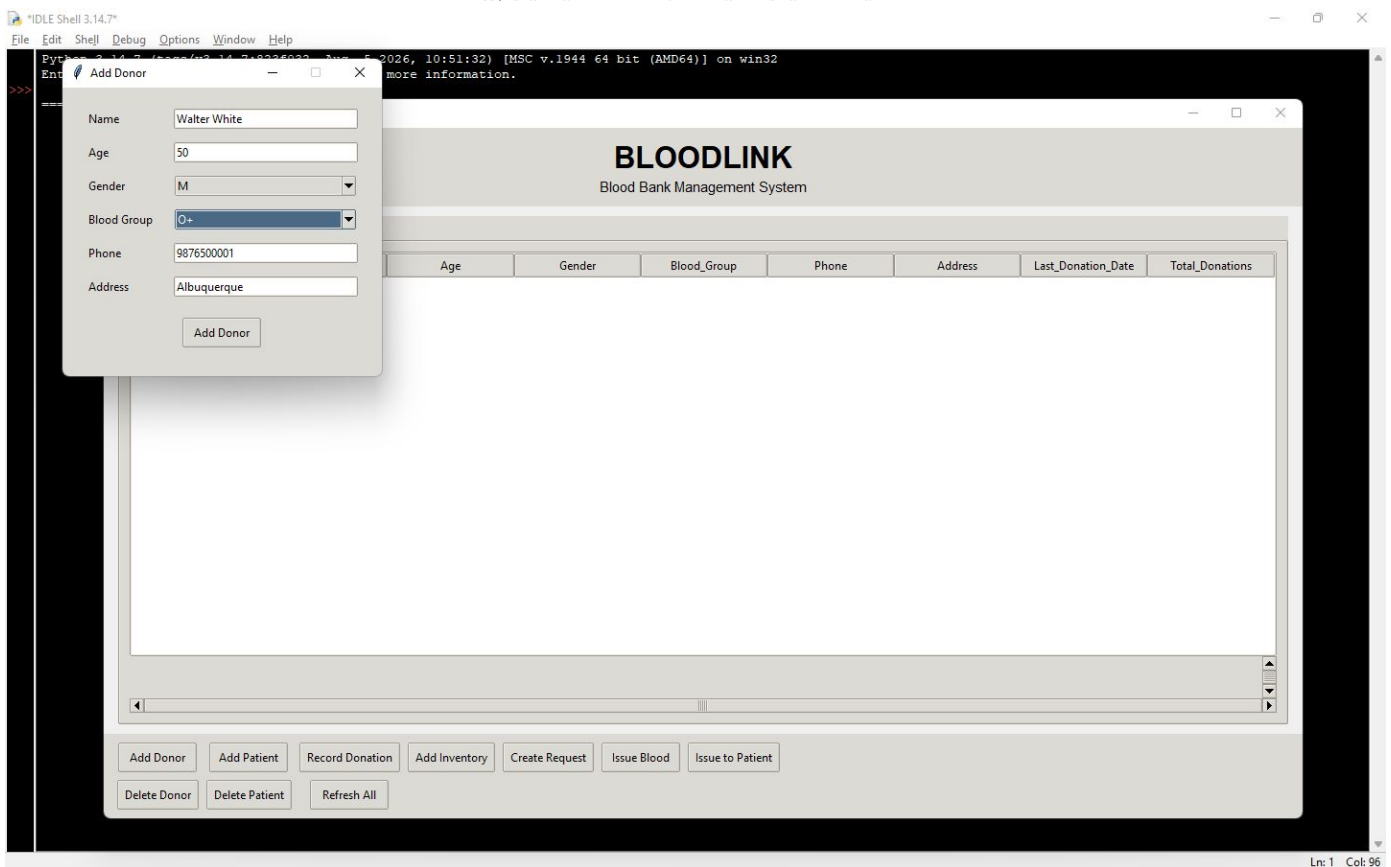
```
root.mainloop()
```

```
if __name__ == "__main__":
    main()
```

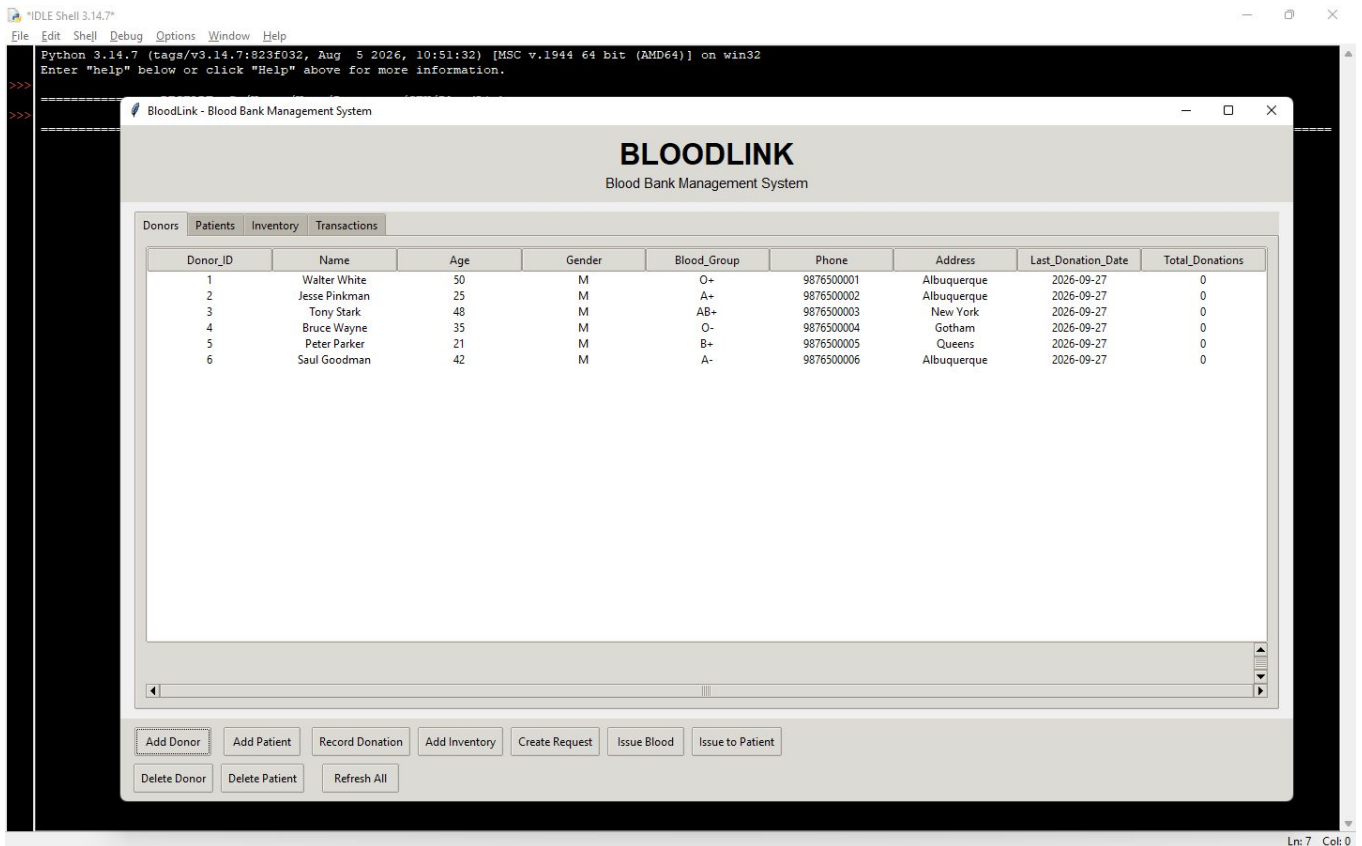
OUTPUT



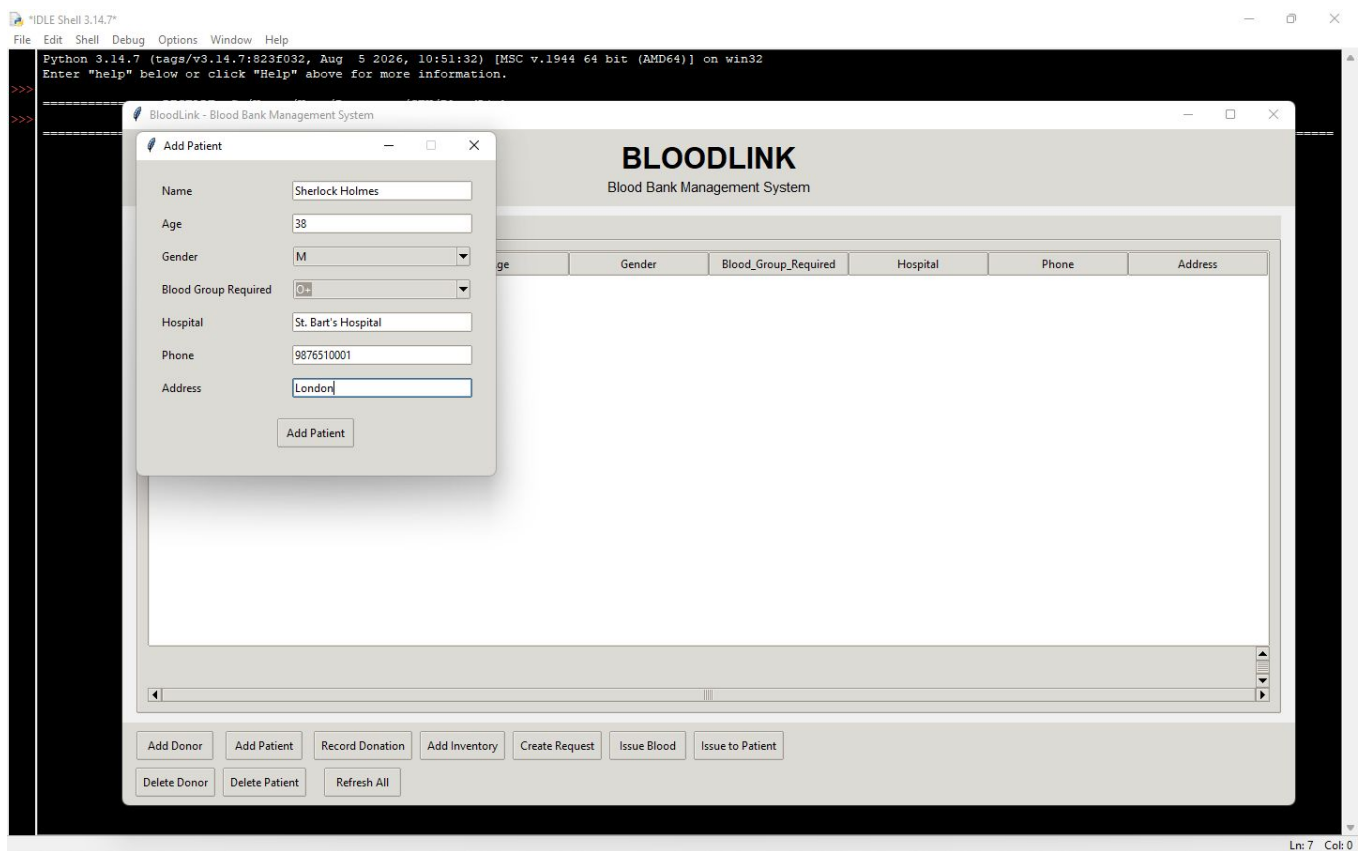
Main BloodLink Interface



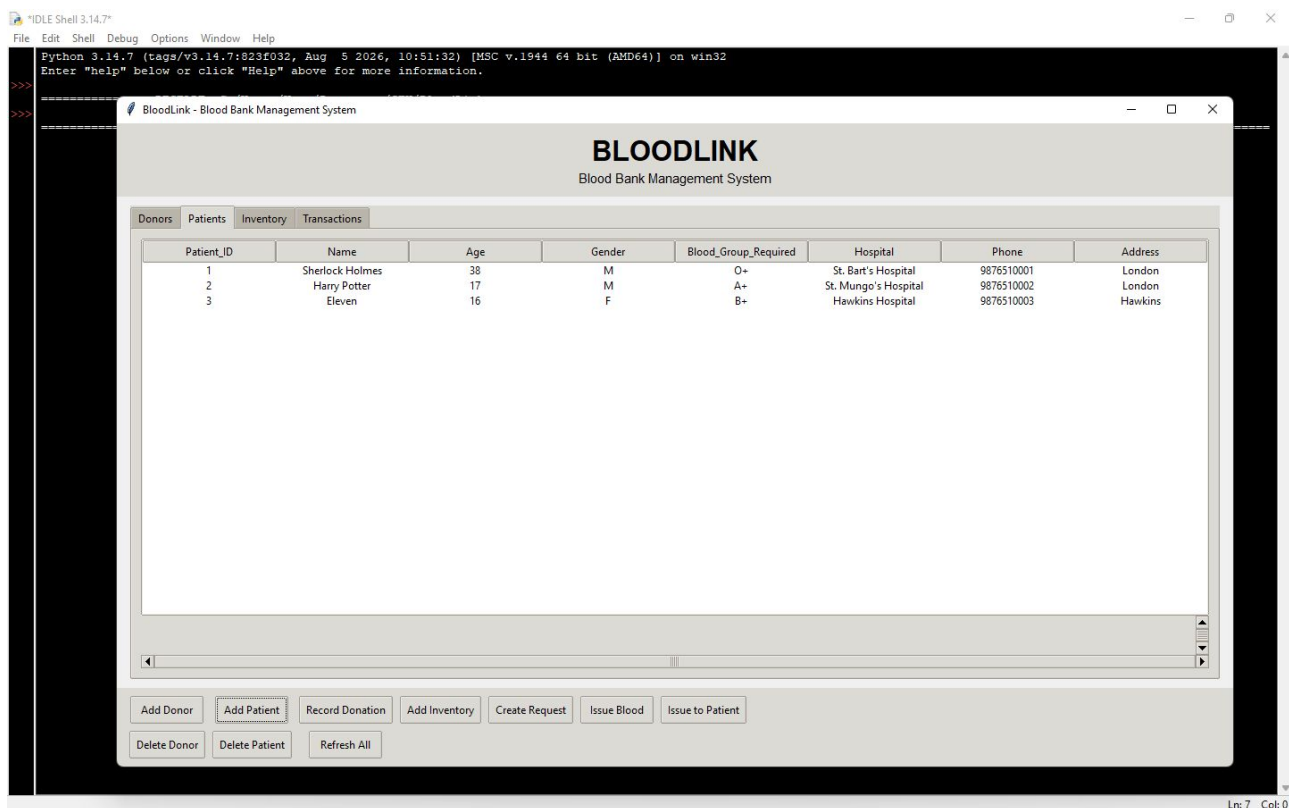
Donor Management



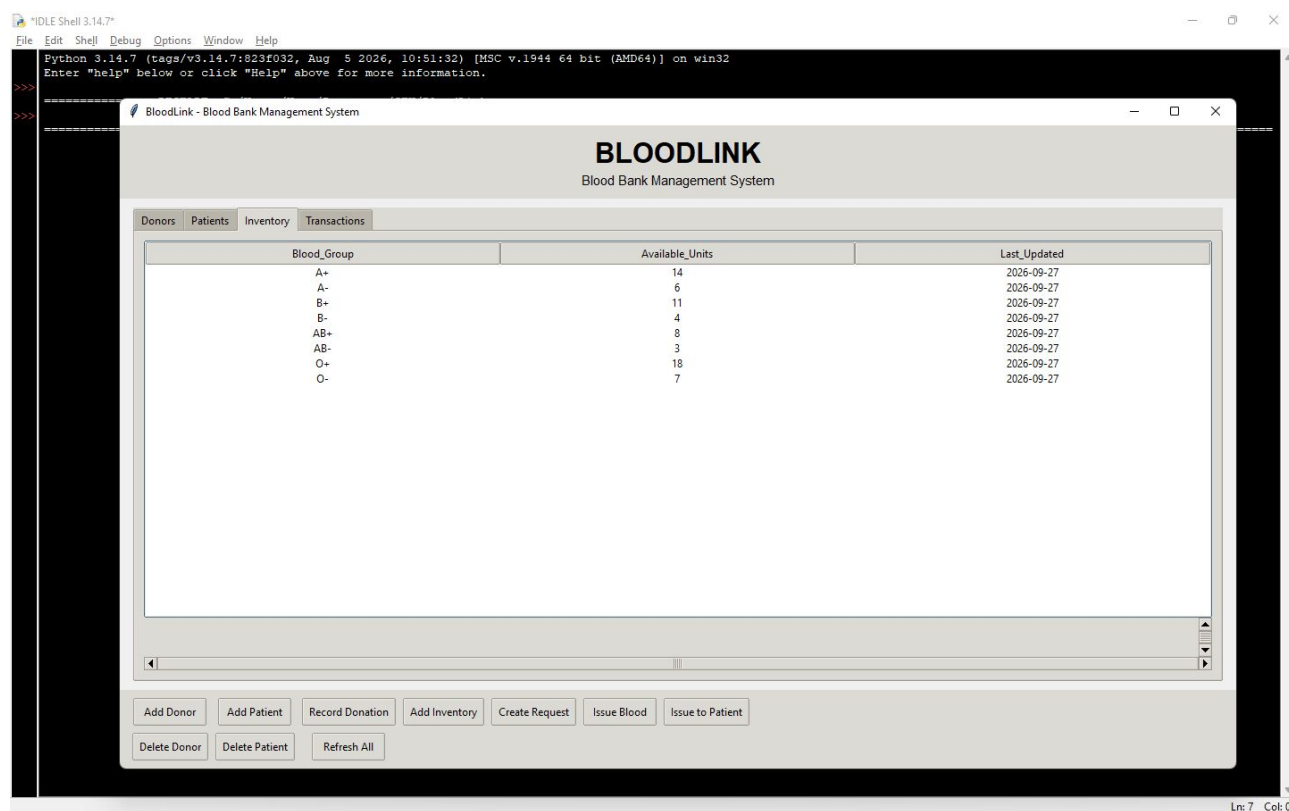
Donor Management



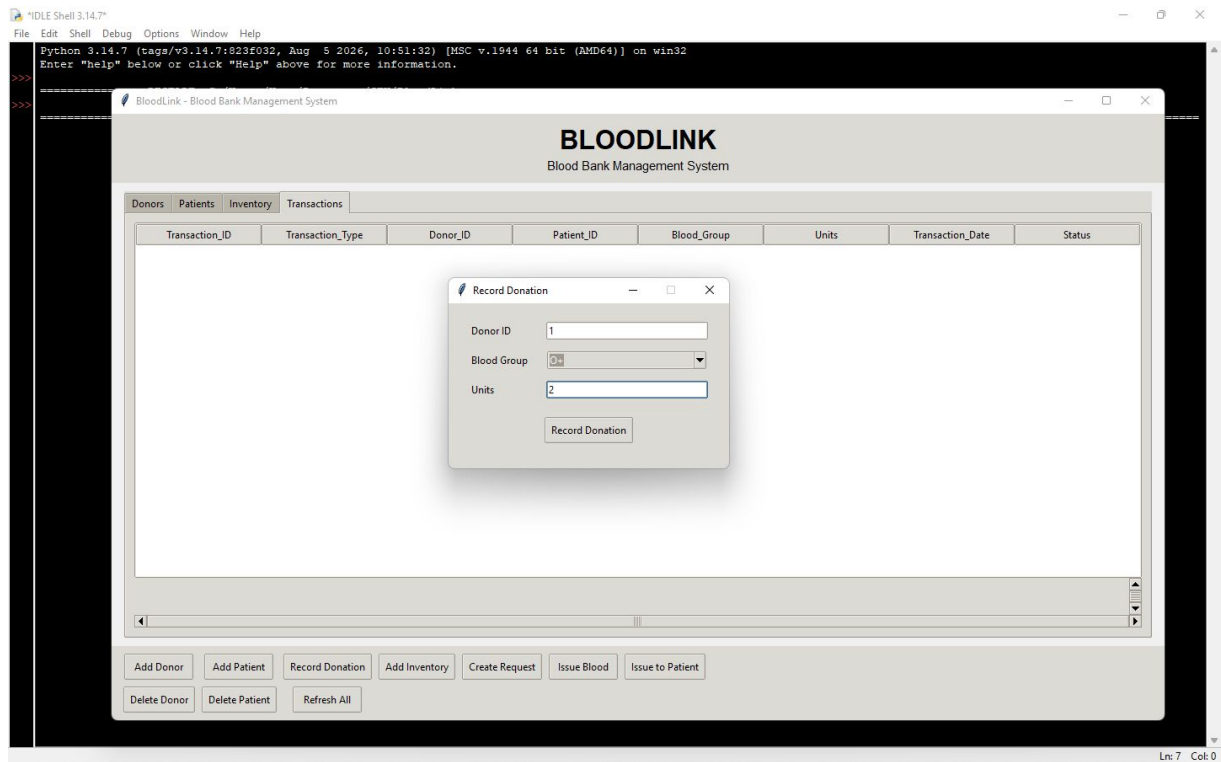
Patient Management



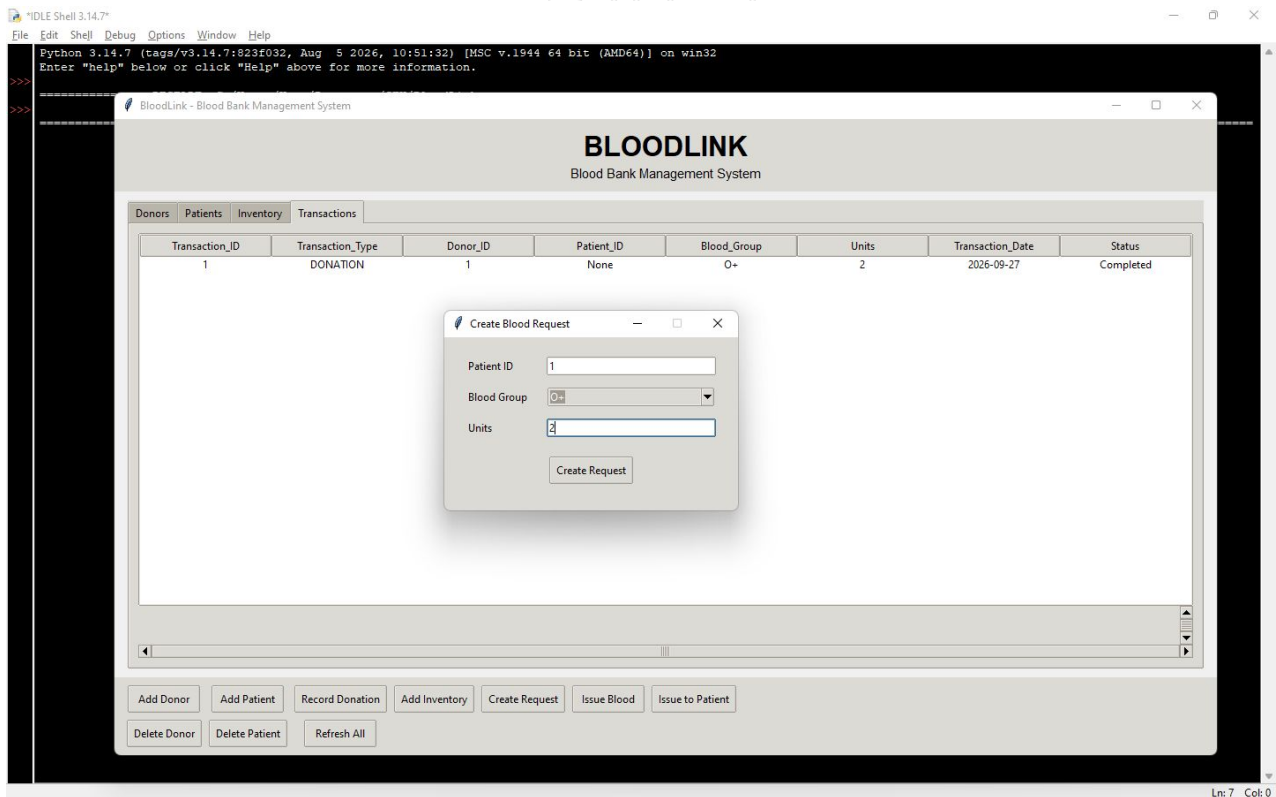
Patient Management



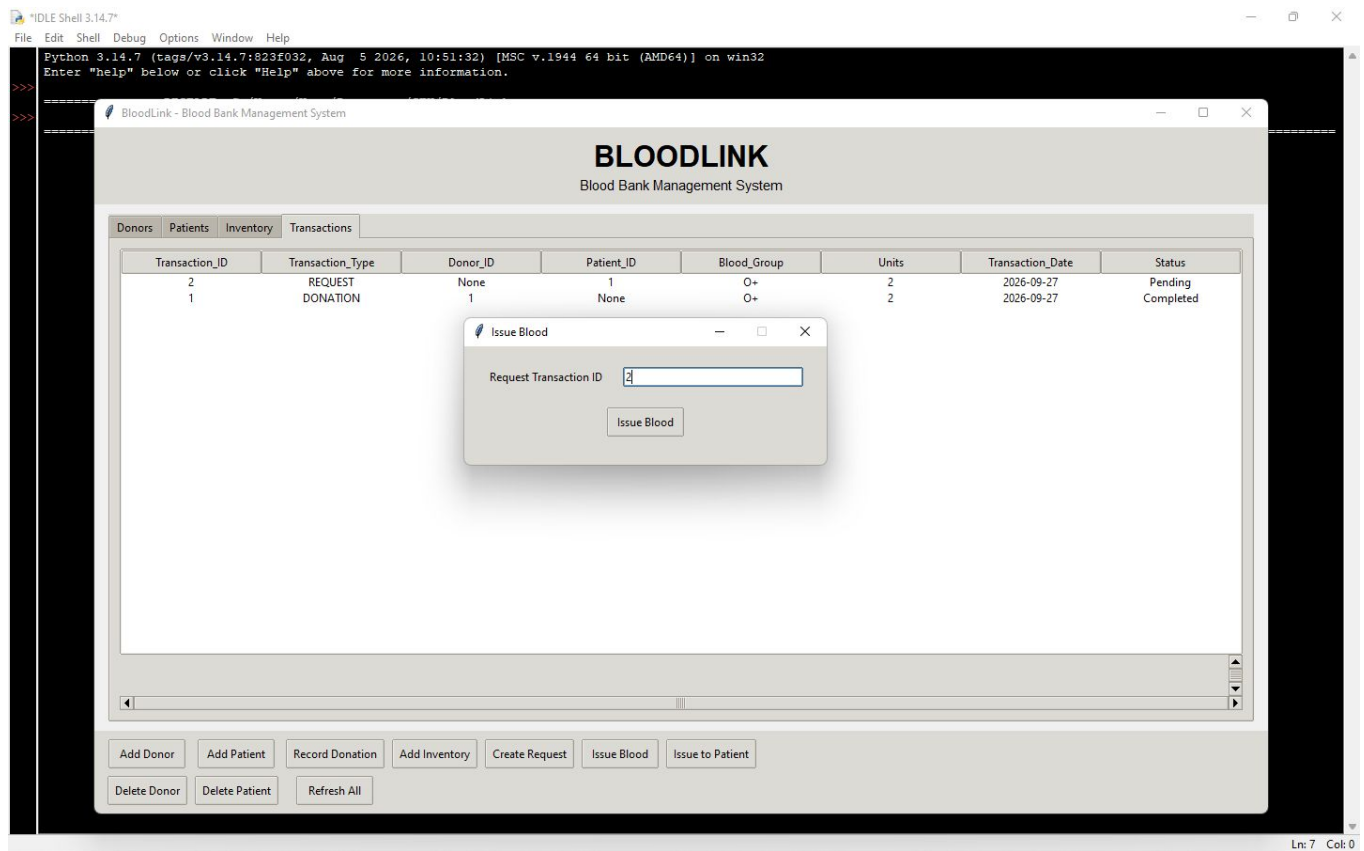
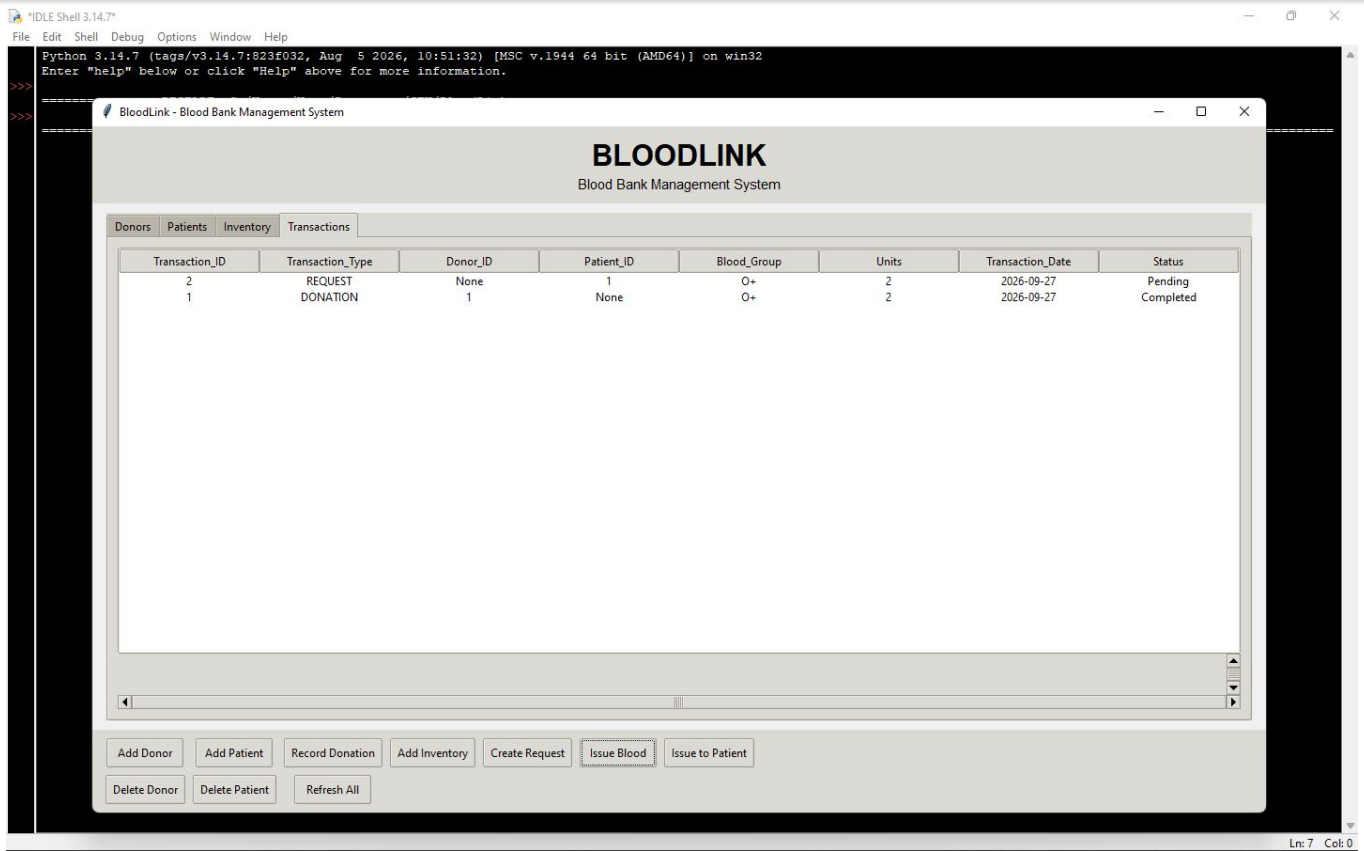
Blood Inventory



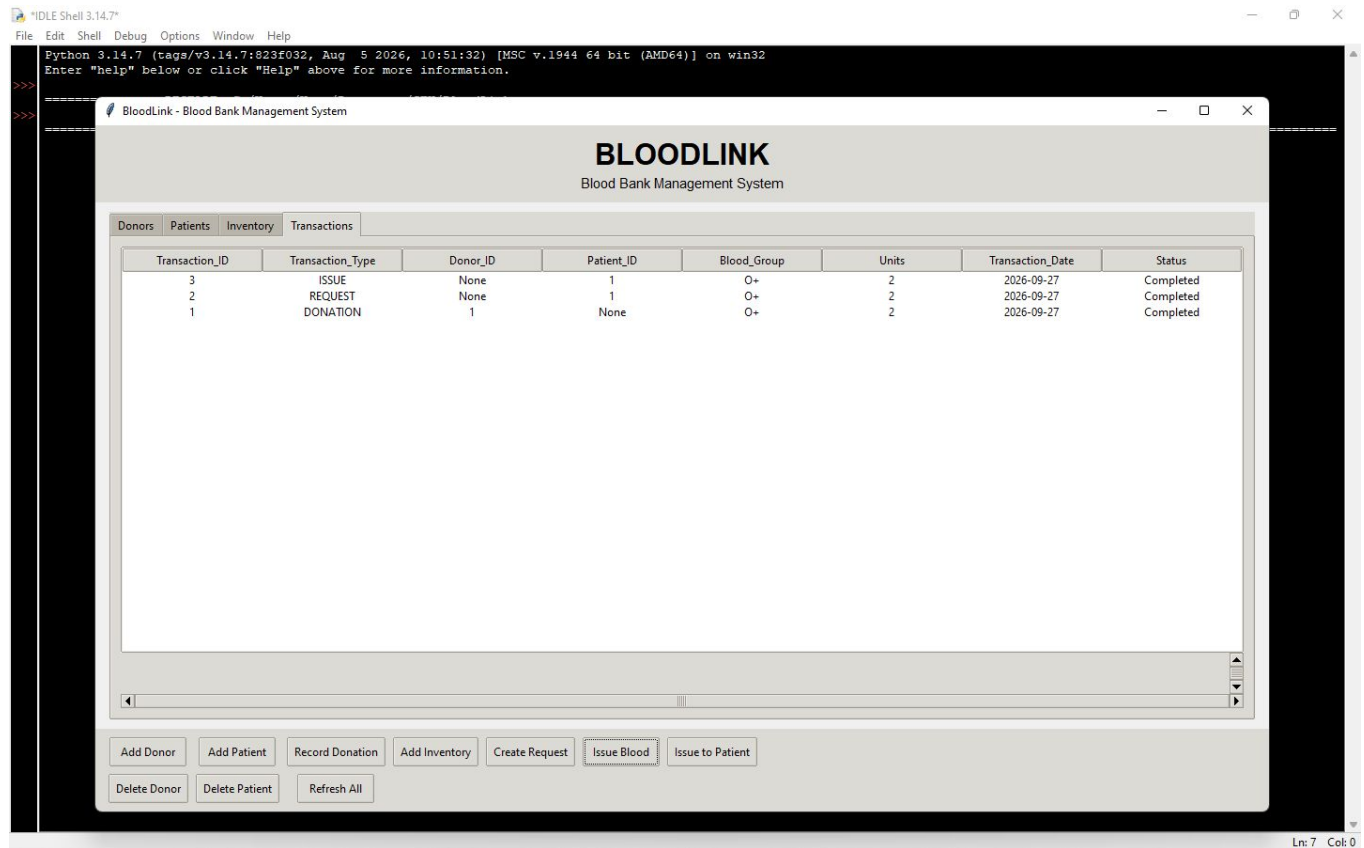
Blood Transactions



Blood Transactions



Blood Transactions



Record Donation

CONCLUSION

BloodLink successfully demonstrates the use of Python programming, SQL database connectivity and Tkinter to create an interactive blood bank management system where donor, patient, inventory and transaction records can be maintained in an organized manner.

This project highlights the importance of integrating Python programming with database management to solve real-life problems. It allows users to add and manage records, maintain blood inventory, record donations, create requests and issue available blood units while keeping a transaction history.

Overall, this project strengthens the understanding of Python and the way it connects to database management while providing a base application that can be extended in the future with features such as multi-user access, online synchronization, advanced reporting, authentication, notifications and role-based permissions.

BIBLIOGRAPHY

1. Computer Science With Python – Class XII, Preeti Arora
2. docs.python.org/3/
3. sqlite.org/docs.html
4. tkinter documentation – docs.python.org/3/library/tkinter.html
5. sqlite3 module documentation – docs.python.org/3/library/sqlite3.html
6. [google.com](https://www.google.com)
7. [youtube.com](https://www.youtube.com)
8. stackoverflow.com